Green World Multi-Role Platform
System Specification

System Specification: Green World Multi-Role Platform

1. Overview

This document describes the architecture and structure of the Green World distribution and e-commerce platform.

The system includes a super admin panel, distributor panel, customer panel, and a driver mobile app.

```
The backend is built with Node.js and MongoDB, with Firebase used for authentication.
```

2. High-Level Architecture

```
Backend: Node.js + Express + MongoDB, Firebase Admin for authentication verification.
```

Super Admin Panel: Pure HTML, CSS, and JavaScript.

Distributor Panel: Pure HTML, CSS, and JavaScript for store and product management.

Customer Panel: Pure HTML, CSS, and JavaScript for shopping, checkout, and order tracking.

Driver App: React Native + Expo mobile application for order delivery management.

2.1 Roles

Super Admin

Distributor

Customer

Driver

Each role is authenticated via Firebase Authentication and authorized via role-based middleware on the backend.

3. Project Structure

3.1 Root Structure

```
backend/ – Node.js + Express + MongoDB backend
```

super-admin-panel/ – HTML super admin panel

distributor-panel/ – HTML distributor panel

customer-panel/ – HTML customer-facing storefront

driver-app/ – React Native + Expo driver mobile app

3.2 Backend Structure (backend/)

```
backend/
  src/
    config/
      database.js       # MongoDB connection
      firebase.js       # Firebase Admin SDK configuration
      env.js            # Environment variables

    models/
      User.js
      Distributor.js
      Store.js
      Product.js
      StoreProduct.js
      Order.js
      Driver.js
      Address.js
      Payment.js

    controllers/
      authController.js
      superAdminController.js
      distributorController.js
      storeController.js
      productController.js
      orderController.js
      customerController.js
      driverController.js

    routes/
      authRoutes.js
      superAdminRoutes.js
      distributorRoutes.js
      storeRoutes.js
      productRoutes.js
      orderRoutes.js
      customerRoutes.js
      driverRoutes.js

    middleware/
      authMiddleware.js
      roleMiddleware.js
      validationMiddleware.js

    utils/
      emailService.js
      notificationService.js
      helpers.js

    app.js
  server.js
  package.json
```

3.3 Super Admin Panel (super-admin-panel/)

```
super-admin-panel/
  index.html
  css/
```

style.css

dashboard.css

responsive.css

```
  js/
    app.js
    distributors.js
    products.js
    orders.js
    drivers.js
  pages/
```

```
    distributors.html
    products.html
    stores.html
    orders.html
    drivers.html
    users.html
    settings.html
  assets/
```

logo.png

3.4 Distributor Panel (distributor-panel/)

```
distributor-panel/
  index.html
  css/
```

style.css

dashboard.css

```
  js/
    app.js
    stores.js
    products.js
    orders.js
  pages/
    stores.html
    store-products.html
    orders.html
    inventory.html
    profile.html
```

3.5 Customer Panel (customer-panel/)

```
customer-panel/
  index.html
  css/
```

style.css

shop.css

cart.css

```
  js/
    app.js
    shop.js
    cart.js
    checkout.js
    orders.js
  pages/
    shop.html
    product-detail.html
    cart.html
    checkout.html
    orders.html
    order-tracking.html
    addresses.html
    profile.html
```

3.6 Driver App (driver-app/)

```
driver-app/
  app/
    (tabs)/
      _layout.tsx
      index.tsx
      orders.tsx
      active-delivery.tsx
      profile.tsx
    login.tsx
```

```
            register.tsx
            +not-found.tsx

          components/
            OrderCard.tsx
            DeliveryStatus.tsx
            MapView.tsx
            AddressCard.tsx
            ProductList.tsx

          services/
```

api.ts

auth.service.ts

order.service.ts

driver.service.ts

firebase.service.ts

4. Database Design (MongoDB)

4.1 Core Collections

users

distributors

stores

products

storeProducts

orders

drivers

addresses

payments

5. Authentication and Authorization

5.1 Authentication Flow

Client uses Firebase Authentication for login/register.

Firebase returns an ID token (JWT).

Client includes Authorization: Bearer <token> in API requests.

Backend verifies the token via Firebase Admin SDK.

5.2 Role-Based Access Control

```
authMiddleware.js verifies Firebase JWT.
roleMiddleware.js checks role permissions.
```

6. Role Capabilities

6.1 Super Admin

- Manage distributors
- Manage product catalog
- View/manage stores and orders
- Manage drivers
- Configure system settings

6.2 Distributor

- Manage stores
- Set store-specific product pricing
- Manage stock/inventory
- View and update store orders

6.3 Customer

- Browse products and stores
- Place orders
- Manage addresses
- Track orders

6.4 Driver

- View assigned deliveries
- Accept/update delivery status
- Use maps/navigation
- View completed deliveries

7. Technology Stack

7.1 Backend

```
- Node.js
- Express.js
```

- MongoDB with Mongoose

- Firebase Admin SDK

7.2 Web Panels

- HTML

- CSS

- Vanilla JavaScript

7.3 Mobile App

- React Native

- Expo

- TypeScript

- Expo Router / React Navigation

- React Native Maps

7.4 Client-Side State & API

- Axios or Fetch

- Zustand

8. Implementation Phases

Phase 1 – Backend Core

Phase 2 – Auth & Roles

Phase 3 – Super Admin Panel

Phase 4 – Distributor & Customer Panels

Phase 5 – Driver App

Phase 6 – Testing & Deployment